

For this lab, I had created a sliding window on both the client and server side. This was done to enforce correct order and create a reliable file transferring program. On the client side, this was done by keeping track of a base sequence number that would only shift once its respective packet sequence number had been acknowledged by the server. Additionally, I had ensured that only a set window size of packets would be sent, that way we really enforce the window size set on client start. For the server, I had ensured that we kept track of expected sequence numbers and base sequence numbers. This was done so that any out of order packets would simply be stored (also acked since I'm using selective repeat) for later writing to the desired file. Once the base sequence number had been received, my write function would write up to the last in order packet before shifting its window and continuing onward.

DESCRIPTIONS:

This file has three separate compartments of source files, each respective to the end it covers: client, common, and server.

Client Directory:

All client related files. The Client.cpp file is where all of the logic behind my client program exists. It reads from a specified file and puts the data into a map containing the respective information on sequence number and the data. C_main.cpp is simply the main for the client.

Common Directory:

Where all universally shared files are stored, used across client and server programs. File_t.cpp is where my reading and writing logic for the client and server are stored. I also have another file: datagram.cpp. This file is where all the byte parsing of packets occurs.

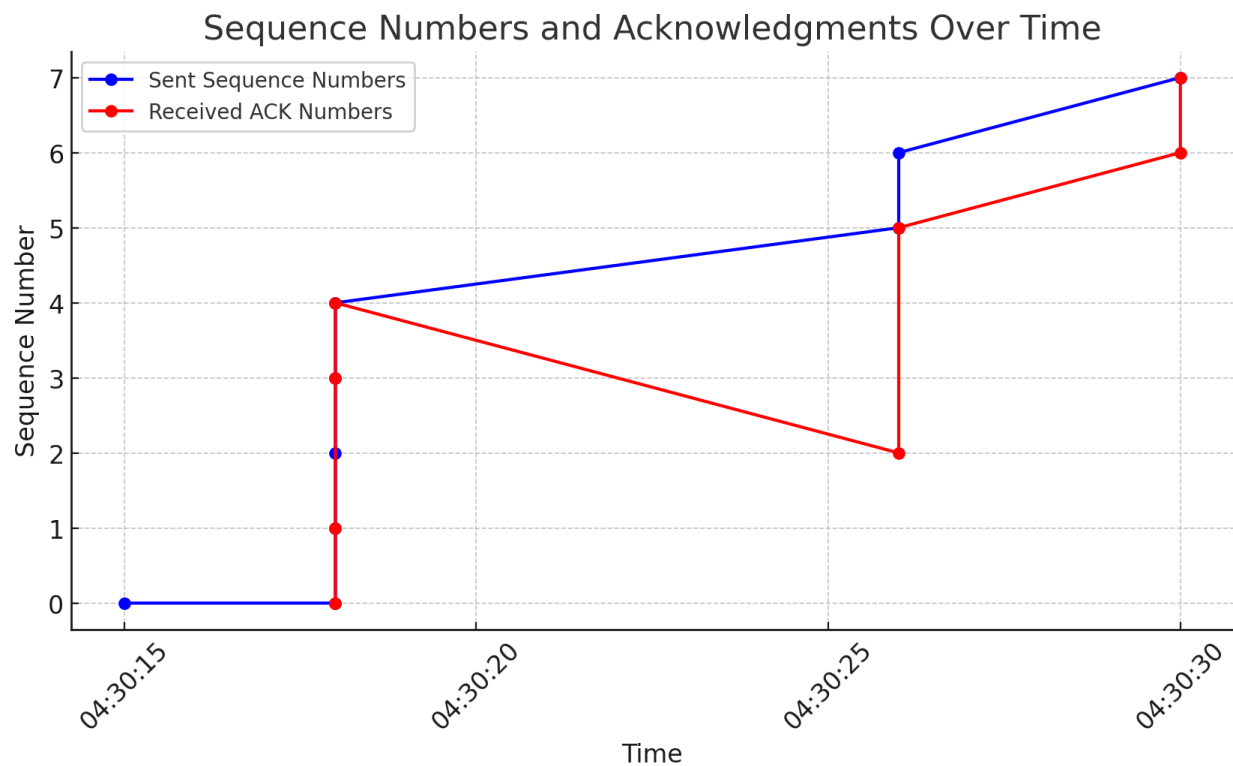
Server Director:

This is where I store all server related files. Conn.cpp (connection) is how I create, view, and modify the client "profile" for each connecting client. I make a map of all clients that are connected and store their respective information. I also have server.cpp, which is where all of my logic behind how the server will send and receive packets is stored.

TESTS:

1. The first test would monitor that my program would complete a single threaded client for one server, checking the window sliding on both ends. This went as expected, considering the compiled file made by the server was an exact copy of the original input file.
2. The second test would check to see if a single client with up to 3 threads would properly maintain thread safety and file integrity across transferring information to 3 separate servers. This also works, considering all three output files were correctly matching to the input file.

3. I tested with multiple clients trying to write to the same file. It's expected that the first client to reach the file will have a "lock" on it and every other client must wait a maximum of 30 seconds. This went as expected, with some clients timing out.
4. I tested if a client could reconnect (i.e. its meta packet wasn't acknowledged even though the server did keep record of the new connection). This went as expected, since I was saving an active file and tied it to a clients port number, the client was able to reconnect and begin the data transfer.
5. I tested multiple clients (4) with 3 threads each. This allowed me to see if each client thread was unique, if the server was able to handle it, and if the threads would exit correctly if any errors occurred. This went exactly as expected. Any threads that terminated wouldn't end the entire procedure, and would instead be summarized in the final return code once all of the threads terminated.



This image shows a client logging rate of ACKed packages vs. DATA packages sent to the server which had a drop rate of 20.